Security Review Report

Nilavan Realtors Next.js Application

> **Assessment type:** Authorized defensive application security review
>
> **Project path:** `lt-nilavan-live`
>
> **Assessment date:** May 8, 2026
>
> **Primary checklist:** OWASP Web Top 10 and OWASP API Security Top 10

Project Explanation

The reviewed application is a marketing and lead-generation website for Nilavan Realtors. It is built with Next.js App Router and React components. The user-facing page presents property information, location/map content, imagery, and a contact form.

The primary dynamic backend surface in the current source tree is `POST /api/sendgrid`, implemented in `app/api/sendgrid/route.ts`. This route accepts contact-form data from `components/contact-section.tsx` and sends an email using SendGrid. The SendGrid API key and recipient address are read from server-side environment variables.

No authentication module, user account model, admin panel, database layer, file upload flow, tenant/project authorization layer, or custom deployment configuration was present in the repository during review. Those missing areas were still considered against OWASP categories, but findings below focus on code-backed risks in the application that exists.

Executive Summary

Severity	Count	Summary
Critical	1	Outdated vulnerable Next.js App Router dependency.
High	1	Public email-sending API has no rate limiting or abuse controls.
Medium	4	Email HTML injection, missing server-side validation, missing browser security headers, and sensitive provider logging.
Low	2	Internal configuration leakage and missing security/audit event logging.
Potential	1	Potential CSS injection in a generic chart component if future data becomes user-controlled.

Scope Notes

- **Authentication/session security:** no login, JWT, password, refresh token, or session handling code was found.
- **Authorization/access control:** no user roles, admin endpoints, object ownership checks, or tenant/project APIs were found.
- **Database security:** no database client, migrations, ORM, raw SQL, or persistence layer was found.
- **File upload security:** no upload/import/download endpoint was found.
- **Main attack surface:** public Next.js routes, frontend rendering, dependencies, configuration, and the SendGrid API route.

Critical Findings

Critical Vulnerable Next.js Version With App Router RSC Exposure

File and line: `package.json:51`

Affected component: Next.js App Router application, including `/api/sendgrid`

Why it is vulnerable: The app pins `next` to `15.1.0`. Official Next.js advisories state that Next.js 15.x App Router applications were affected by React Server Components vulnerabilities, including CVE-2025-66478. For the 15.1.x release line, the RCE advisory was fixed in `15.1.9`, and later RSC denial-of-service/source-exposure fixes require `15.1.11`.

Realistic impact: Depending on runtime and exposure, an unpatched deployment can be vulnerable to remote code execution, denial of service, or source disclosure.

Safe proof of concept: Do not run exploit payloads. Confirm the affected version defensively:

```
npm ls next
```

Recommended fix: Upgrade Next.js to a patched version, then rebuild and redeploy.

```
npm install next@15.1.11
npm run build
npm audit
```

Test case: Verify `npm ls next` reports `15.1.11` or newer patched release, the app builds, and `npm audit` remains clean.

High Findings

High Public Email API Has No Rate Limit, Abuse Control, or Bot Protection

File and line: `app/api/sendgrid/route.ts:4`

Affected function/API: `POST /api/sendgrid`

Why it is vulnerable: The route is publicly reachable and sends email through SendGrid for every valid request path. It has no IP-based throttling, CAPTCHA/Turnstile, honeypot, origin checks, or server-side abuse threshold.

Realistic impact: Attackers can automate submissions to spam the recipient, exhaust SendGrid quota, damage sender reputation, and create operational noise.

Safe proof of concept: In local development, send repeated harmless POST requests and observe that the server does not return `429 Too Many Requests`.

```
Invoke-WebRequest -Method POST http://localhost:3000/api/sendgrid `
  -ContentType "application/json" `
  -Body '{"name":"test","email":"test@example.com","phone":"1234567890","message":"rate
limit test"}'
```

Recommended fix: Add a production-grade shared rate limiter before calling `sendgrid.send`. Use a shared store such as Redis, Vercel KV, or Upstash so limits work across server instances.

```
const ip = req.headers.get("x-forwarded-for")?.split(",")[0]?.trim() ?? "unknown";
// Check a shared store: 3 submissions per 10 minutes per IP.
if (isRateLimited) {
  return NextResponse.json({ error: "Too many requests" }, { status: 429 });
}
```

Test case: Send five requests from the same IP in one minute. Requests beyond the configured threshold should return `429` and should not call SendGrid.

Medium Findings

Medium HTML Injection in Email Template

File and lines: `app/api/sendgrid/route.ts:29-47`, especially `39-42`

Affected function/API: `POST /api/sendgrid`, email HTML body

Why it is vulnerable: User-controlled values `name`, `email`, `phone`, and `message` are directly inserted into an HTML email template without escaping.

Realistic impact: An attacker can alter the email body, add phishing links, hide legitimate labels, or inject misleading HTML content into emails viewed by staff.

Safe proof of concept: Submit harmless HTML such as `<b>Injected heading</b>`. If the received email renders bold text instead of literal text, the field is injectable.

Recommended fix: Escape all user-controlled values before interpolating into HTML, or use a templating library that auto-escapes output.

```
const escapeHtml = (value: unknown) =>
  String(value ?? "").replace(/[<>&"']/g, (char) =>
    ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[char]!)
  );
```

Test case: Submit `<b>test</b>` and verify the email displays the literal string rather than rendered HTML.

Medium Missing Server-Side Input Validation and Size Constraints

File and lines: `app/api/sendgrid/route.ts:6-7`; frontend source `components/contact-section.tsx:18-32`

Affected function/API: Contact form and `POST /api/sendgrid`

Why it is vulnerable: Browser fields are marked `required`, but direct API calls bypass client-side controls. The server accepts any JSON shape and does not validate data types, email format, phone format, field lengths, or message size.

Realistic impact: Invalid or oversized payloads can produce SendGrid errors, log noise, abuse, unexpected email content, and higher processing cost.

Safe proof of concept: Send an invalid email or oversized message to the local API. The current handler reaches email construction instead of rejecting the request early.

Recommended fix: Validate request bodies server-side with a schema library such as Zod.

```
const ContactSchema = z.object({
  name: z.string().trim().min(1).max(80),
  email: z.string().trim().email().max(254),
  phone: z.string().trim().regex(/^[0-9+\-\s()]{7,20}$/),
  message: z.string().trim().min(1).max(1000),
});

const parsed = ContactSchema.safeParse(await req.json());
if (!parsed.success) {
  return NextResponse.json({ error: "Invalid request" }, { status: 400 });
}
```

Test case: Missing fields, invalid email, non-string values, and a 50 KB message should return `400` and should not send email.

Medium Missing Security Headers and Content Security Policy

File and line: No `next.config.*` or `middleware.ts` was present.

Affected component: Entire web application

Why it is vulnerable: The app does not define defensive browser headers such as Content Security Policy, frame protection, referrer policy, permissions policy, or MIME sniffing protection.

Realistic impact: Missing headers increase the blast radius of XSS, clickjacking, data leakage via referrer headers, and unwanted browser feature exposure.

Safe proof of concept: Request the homepage and inspect response headers. The expected security headers are absent unless added by hosting infrastructure.

Recommended fix: Add headers in `next.config.ts` or equivalent deployment configuration.

```
const nextConfig = {
  async headers() {
    return [{
      source: "/(.*)",
      headers: [
        { key: "Content-Security-Policy", value: "default-src 'self'; img-src 'self'
data:; style-src 'self' 'unsafe-inline'; script-src 'self'; frame-ancestors 'none'; base-
uri 'self'; form-action 'self'" },
        { key: "X-Content-Type-Options", value: "nosniff" },
        { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
        { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" }
      ]
    }];
  }
};
export default nextConfig;
```

Test case: Run the app and verify these headers appear on `/` and API responses where appropriate.

Medium Sensitive Third-Party Error Details Logged

File and lines: `app/api/sendgrid/route.ts:52-58`

Affected function/API: SendGrid error handling

Why it is vulnerable: The handler logs the full error and SendGrid response body. Provider errors can include submitted contact data, request metadata, recipient details, or diagnostics not intended for broad log access.

Realistic impact: Contact-form PII and provider metadata may be exposed in development logs or centralized production logging systems.

Safe proof of concept: Use an invalid SendGrid key in a development environment and submit test PII. Inspect logs for full provider response data.

Recommended fix: Log minimal structured metadata and avoid full request bodies or provider responses.

```
console.error("SendGrid send failed", {
  route: "/api/sendgrid",
  status: errorStatus,
  event: "contact_email_send_failed"
});
```

Test case: Force a SendGrid failure and confirm logs do not contain submitted `name`, `email`, `phone`, or `message`.

Low Findings

Low Internal Configuration State Leaked in API Errors

File and lines: `app/api/sendgrid/route.ts:12-20`

Affected function/API: `POST /api/sendgrid`

Why it is vulnerable: The API response tells callers whether `SENDGRID_API_KEY` or `SENDGRID_TO_EMAIL` is missing.

Realistic impact: This is a small information disclosure that helps attackers fingerprint deployment state and misconfiguration.

Safe proof of concept: Temporarily unset one variable in local development and observe the specific error response.

Recommended fix: Return a generic service error to clients while logging sanitized details server-side.

```
return NextResponse.json({ error: "Service unavailable" }, { status: 503 });
```

Test case: Unset each variable locally and verify callers receive only a generic error.

Low Missing Security Event and Abuse Logging

File and line: `app/api/sendgrid/route.ts:50`

Affected function/API: `POST /api/sendgrid`

Why it is vulnerable: The application does not create structured logs for successful sends, validation failures, rate-limit events, or repeated abuse patterns.

Realistic impact: Spam or abuse may not be detected until SendGrid quota, sender reputation, or staff inbox quality is affected.

Safe proof of concept: Submit valid and invalid requests locally and confirm there is no structured security event trail.

Recommended fix: Add minimal structured audit events with route, outcome, timestamp, and hashed IP. Do not log full message content.

```
console.info("security_event", {
  event: "contact_form_submit",
  outcome: "accepted",
  route: "/api/sendgrid"
});
```

Test case: Submit valid, invalid, and throttled requests and confirm sanitized events are emitted.

Potential Finding

Potential CSS Injection in Generic Chart Component

File and lines: `components/ui/chart.tsx:80-98`

Affected component: `ChartStyle`

Why it is vulnerable: The component uses `dangerouslySetInnerHTML` to inject CSS variables from a chart config. No current user-controlled use was found, so this is marked potential rather than confirmed exploitable.

Realistic impact if exposed later: If future chart configuration is user-controlled, attackers may inject unexpected CSS that manipulates the UI or performs limited CSS-based data exposure patterns.

Safe proof of concept: In a local component test, pass a color value such as `red; background:url(https://example.com)` and verify whether it reaches the style tag.

Recommended fix: Restrict chart color values to known-safe formats before insertion.

```
const isSafeColor = (value: string) =>
  /^#([0-9a-f]{3}|[0-9a-f]{6}|[0-9a-f]{8})$/i.test(value) ||
  /^hsl\([\d\s.%,-]+\)$/i.test(value);
```

Test case: Unsafe CSS fragments should be rejected and should not appear inside the generated style tag.

Dependency Risks

- **Confirmed risk:** `next@15.1.0` is below the patched versions described in the official Next.js advisories for App Router RSC vulnerabilities.
- **Audit result:** `npm audit --json` returned zero known vulnerable dependencies in the local lockfile at review time.
- **Outdated packages:** `npm outdated` showed newer releases for several packages, including `next`, `react`, `react-dom`, `zod`, and `@sendgrid/mail`. Prioritize security-patched framework updates first, then plan broader compatibility upgrades.

Configuration Risks

- No `.env` files were found in the repository.
- No custom CORS configuration was found. Same-origin browser rules apply, but direct API calls remain possible.
- No `next.config.*`, `middleware.ts`, Dockerfile, nginx config, or Vercel config was found, so deployment-level headers and hardening are not represented in source control.

- No debug flag or hardcoded credentials were found in the application source.

Recommended Fix Priority

1. Upgrade `next` to `15.1.11` or a tested newer patched release, rebuild, and redeploy.
2. Add server-side Zod validation, length limits, and HTML escaping to `/api/sendgrid`.
3. Add rate limiting and bot protection before calling SendGrid.
4. Add security headers through `next.config.ts` or deployment configuration.
5. Sanitize error logging and add minimal structured security events.
6. Keep dependency checks in CI using `npm audit` and scheduled upgrade review.

Security Test Cases

Area	Test	Expected Result
Dependency	Run `npm ls next`, `npm audit`, and `npm run build`.	Next.js is patched, audit is clean, build succeeds.
Rate limiting	Send five rapid contact requests from the same IP.	Requests above the threshold return `429` and do not send email.
Validation	Submit missing fields, invalid email, wrong JSON types, and oversized messages.	Server returns `400`; SendGrid is not called.
HTML injection	Submit `<b>test</b>` in each field.	Email displays literal escaped text, not rendered HTML.
Error hygiene	Unset SendGrid environment variables locally.	Client sees a generic service error; server logs are sanitized.
Security headers	Inspect response headers for `/`.	CSP, frame protection, referrer policy, permissions policy, and nosniff are present.
Audit logging	Submit valid, invalid, and throttled requests.	Sanitized structured events are recorded without full PII.

References

- OWASP Web Top 10: `https://owasp.org/www-project-top-ten/`
- OWASP API Security Top 10: `https://owasp.org/API-Security/`
- Next.js CVE-2025-66478 advisory: `https://nextjs.org/blog/CVE-2025-66478`
- Next.js December 11, 2025 security update: `https://nextjs.org/blog/security-update-2025-12-11`